

리눅스 운영과 배포

LFCS 도메인 2 (배점 25%) — 부팅 프로세스, systemd, 작업 스케줄링, 로그 분석, 프로세스·리소스 관리, 커널 파라미터, SELinux, 셸 스크립트

목차

1. 부팅 프로세스
2. systemd 서비스 관리
3. 유닛 파일 작성과 재정의
4. 타깃과 부팅 모드
5. 작업 스케줄링 — cron, at, systemd timer
6. 로그 분석 — journald와 rsyslog
7. 프로세스 관리
8. 시스템 리소스 모니터링
9. 패키지 관리
10. 커널 파라미터와 모듈
11. SELinux
12. 셸 스크립트 기초
13. 실습 체크리스트 / 자주 하는 실수

1. 부팅 프로세스

1.1 전체 흐름

```
전원 인가
↓
① 펌웨어 (BIOS / UEFI)   하드웨어 초기화, 부팅 장치 선택
↓
② 부트로더 (GRUB2)       커널과 initramfs를 메모리에 적재
↓
③ 커널                   하드웨어 인식, 루트 파일시스템 마운트
↓
④ initramfs              실제 루트를 마운트하는 데 필요한 드라이버 제공
↓
⑤ systemd (PID 1)        유닛 의존성 해석 후 서비스 기동
↓
⑥ 타깃 도달             multi-user.target / graphical.target
```

장애 지점을 좁힐 때 이 순서가 그대로 진단 순서가 됩니다. GRUB 메뉴가 안 보이면 ①~②, 커널 패닉이면 ③~④, 로그인은 되는데 서비스가 없으면 ⑤~⑥ 문제입니다.

1.2 GRUB2

```
sudo vim /etc/default/grub
```

```
GRUB_TIMEOUT=5
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
GRUB_DEFAULT=0
```

```
sudo update-grub          # Debian/Ubuntu
sudo grub2-mkconfig -o /boot/grub2/grub.cfg # RHEL 계열
```

/boot/grub/grub.cfg 를 직접 편집하지 마세요. update-grub 이 자동 생성하는 파일이라 다음 커널 업데이트 때 덮어써집니다. 항상 /etc/default/grub 이나 /etc/grub.d/ 를 고친 뒤 재생성합니다.

부팅 시 임시 커널 파라미터

GRUB 메뉴에서 `e` 를 눌러 `linux` 줄 끝에 추가한 뒤 `Ctrl+X` 로 부팅합니다. 이번 부팅에만 적용됩니다.

파라미터	효과
<code>single</code> 또는 <code>1</code>	싱글 유저 모드
<code>systemd.unit=rescue.target</code>	레스큐 모드
<code>init=/bin/bash</code>	셸로 직접 진입 — root 비밀번호 복구
<code>nomodeset</code>	그래픽 드라이버 문제 회피

root 비밀번호를 잊었을 때가 대표적인 실기 시험 문제입니다. 흐름은 이렇습니다: GRUB에서 `rd.break` 또는 `init=/bin/bash` → 루트를 `rw`로 `remount` → `passwd root` → (SELinux 환경이면) `touch /.autorelabel` → 재부팅. 마지막 단계를 빠뜨리면 SELinux 컨텍스트가 어긋나 로그인 안 됩니다.

1.3 부팅 분석

```
systemd-analyze          # 총 부팅 시간
systemd-analyze blame    # 서비스별 소요 시간 순위
systemd-analyze critical-chain # 임계 경로
journalctl -b            # 이번 부팅 로그 전체
journalctl -b -1         # 직전 부팅 로그
journalctl -p err -b     # 이번 부팅의 오류만
dmesg -T | less          # 커널 메시지 (사람이 읽는 시각)
```

재부팅 후 문제가 생겼다면 `journalctl -b -1 -p err` 가 첫 명령입니다. 직전 부팅에서 어떤 오류가 있었는지 바로 보여줍니다. 서버가 예기치 않게 재부팅 됐을 때 원인을 찾는 출발점이기도 합니다.

2. systemd 서비스 관리

2.1 기본 명령

```
systemctl status nginx          # 상태 + 최근 로그
sudo systemctl start nginx
sudo systemctl stop nginx
sudo systemctl restart nginx    # 중단 발생
sudo systemctl reload nginx     # 설정만 재적용 (무중단)
sudo systemctl reload-or-restart nginx

sudo systemctl enable nginx     # 부팅 시 자동 시작
sudo systemctl disable nginx
sudo systemctl enable --now nginx # 활성화 + 즉시 시작

sudo systemctl mask nginx       # 완전 차단 (수동 시작도 불가)
sudo systemctl unmask nginx
```

disable 과 **mask** 의 차이가 자주 나옵니다. **disable** 은 자동 시작만 막고 수동 **start** 는 가능합니다. **mask** 는 유닛을 **/dev/null** 로 링크해 어떤 방법으로도 시작되지 않게 만듭니다. 다른 서비스의 의존성으로도 기동되지 않습니다.

"서비스를 켜는데 자꾸 살아난다"면 다른 유닛이 의존성으로 끌어올리고 있는 것이고, 이때 **mask** 가 답입니다.

2.2 상태 조회

```
systemctl list-units --type=service    # 활성 서비스
systemctl list-units --type=service --all # 비활성 포함
systemctl list-unit-files --state=enabled # 자동 시작 목록
systemctl --failed                     # 실패한 유닛만 — 점검 시 필수
systemctl is-active nginx              # 스크립트용 (종료 코드 반환)
systemctl is-enabled nginx
systemctl cat nginx                    # 유닛 파일 내용 + 재정의
systemctl show nginx                   # 모든 속성
systemctl list-dependencies nginx      # 의존성 트리
```

2.3 status 출력 읽기

```
● nginx.service - A high performance web server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; ...)
   Active: active (running) since Wed 2026-09-02 10:00:00 KST; 2h ago
 Main PID: 1234 (nginx)
    Tasks: 3 (limit: 4915)
   Memory: 12.3M
```

Active 값	의미
active (running)	정상 동작 중
active (exited)	한 번 실행되고 끝남 (oneshot). 정상일 수 있음
inactive (dead)	중지됨
failed	비정상 종료. 원인 조사 필요
activating	기동 중 (오래 머물면 문제)

Loaded 줄의 **enabled** / **disabled** 는 **자동 시작 여부**이고, **Active** 는 **현재 실행 상태**입니다. 둘은 별개입니다.

3. 유닛 파일 작성과 재정의

3.1 위치 우선순위

경로	용도	우선순위
/lib/systemd/system/	패키지가 설치한 원본	낮음
/etc/systemd/system/	관리자가 만든 것	높음
/etc/systemd/system/서비스.d/	부분 재정의 (drop-in)	가장 높음

3.2 직접 작성하기

```
sudo vim /etc/systemd/system/myapp.service
```

```
[Unit]
Description=My Application Server
Documentation=https://example.com/docs
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=appsvc
Group=appsvc
WorkingDirectory=/srv/myapp
Environment="NODE_ENV=production"
EnvironmentFile=-/etc/myapp/env
ExecStart=/usr/bin/node /srv/myapp/server.js
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=5
LimitNOFILE=65535

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload      # 유닛 파일 변경 후 필수
sudo systemctl enable --now myapp
```

유닛 파일을 고쳤으면 반드시 **daemon-reload** 를 실행하세요. 이걸 빠뜨리면 systemd가 이전 내용을 그대로 쓰고 있어서 "분명히 수정했는데 반응이 안 된다"는 상황이 됩니다. 가장 흔한 실수 중 하나입니다.

3.3 주요 지시어

[Unit] 섹션

지시어	의미
After=	순서만 지정. 해당 유닛이 없어도 기동됨
Requires=	강한 의존. 해당 유닛 실패 시 함께 실패
Wants=	약한 의존. 실패해도 계속 진행 (권장)
Conflicts=	동시 실행 불가

After 는 **순서**, Requires / Wants 는 **의존성**입니다. 보통 둘을 같이 씁니다.

Type 값

Type	동작	용도
simple	ExecStart가 주 프로세스. 기본값	포그라운드로 도는 앱
forking	프로세스가 데몬화(fork)됨	전통적 데몬. PIDFile= 필요
oneshot	한 번 실행 후 종료	초기화 스크립트
notify	준비 완료를 systemd에 알림	정확한 기동 감지 필요 시

Type을 잘못 고르면 서비스가 정상인데도 failed 로 표시되거나, 반대로 죽었는데 active 로 보입니다. 애플리케이션이 포그라운드로 돌면 simple , 스스로 백그라운드로 넘어가면 forking 입니다. 최근 애플리케이션은 대부분 simple 로 쓰는 것이 맞습니다.

Restart 정책

값	동작
no	재시작 안 함 (기본값)
on-failure	비정상 종료 시만. 가장 실용적
always	정상 종료해도 재시작
on-abnormal	시그널·타임아웃에 의한 종료 시

Restart=always 에 RestartSec 이 너무 짧으면 재시작 루프에 빠져 CPU를 태웁니다. StartLimitBurst=5 , StartLimitIntervalSec=60 으로 한도를 두세요. 한도를 넘으면 systemd가 포기하고 failed 로 남깁니다 — 이게 오히려 정상 동작이며, 무한 재시도보다 문제 인지가 빠릅니다.

3.4 드롭인으로 부분 재정의

패키지가 제공한 유닛 파일을 통째로 복사하지 말고, 바꿀 항목만 덮어쓰는 것이 올바른 방식입니다.

```
sudo systemctl edit nginx
```

```
[Service]
LimitNOFILE=65535
Restart=on-failure
```

/etc/systemd/system/nginx.service.d/override.conf 가 자동 생성됩니다. 패키지가 업데이트되어도 이 재정의는 유지됩니다.

```
systemctl cat nginx          # 원본 + 재정의를 함께 확인
sudo systemctl revert nginx  # 재정의 제거하고 원상 복구
```

ExecStart 처럼 목록형 지시어를 재정의할 때는 먼저 빈 값으로 초기화해야 합니다.

```
[Service]
ExecStart=
ExecStart=/usr/bin/new-command
```

그냥 추가하면 두 개가 모두 실행되려다 오류가 납니다.

4. 타깃과 부팅 모드

systemd의 타깃은 예전 런레벨을 대체합니다.

타깃	런레벨	설명
poweroff.target	0	종료
rescue.target	1	단일 사용자, 최소 서비스
multi-user.target	3	서버 기본. 네트워크 + CLI
graphical.target	5	GUI 포함
reboot.target	6	재부팅
emergency.target	-	최소한의 셸. 루트가 ro로 마운트

```
systemctl get-default          # 기본 타깃 확인
sudo systemctl set-default multi-user.target # 서버는 GUI 불필요
sudo systemctl isolate rescue.target      # 즉시 전환
```

서버에서 기본 타깃이 graphical.target 이면 불필요한 메모리와 CPU를 씁니다. multi-user.target 으로 바꾸면 자원이 절약되고 공격 표면도 줄어듭니다.

5. 작업 스케줄링 — cron, at, systemd timer

5.1 cron 문법

```
┌── 분 (0-59)
├── 시 (0-23)
├── 일 (1-31)
├── 월 (1-12)
├── 요일 (0-7, 0과 7은 일요일)
└── * * * * * 실행할 명령
```

표기	의미
*	모든 값
*/5	5마다 (0,5,10...)
1,15	1과 15
1-5	1부터 5까지

예시

```
0 3 * * *    매일 새벽 3시
*/10 * * * * 10분마다
0 */6 * * *   6시간마다
30 2 * * 0    매주 일요일 02:30
0 0 1 * *     매월 1일 자정
0 9 * * 1-5   평일 오전 9시
```

5.2 crontab 사용

```
crontab -e          # 내 crontab 편집
crontab -l          # 목록
crontab -r          # 전체 삭제 (주의!)
sudo crontab -e -u jihoon # 특정 사용자
```

시스템 crontab

```
sudo vim /etc/crontab
```

```
# 사용자 필드가 추가됨
0 3 * * * root /usr/local/bin/backup.sh
```

디렉터리 방식도 있습니다: `/etc/cron.d/` (개별 파일), `/etc/cron.daily/`, `/etc/cron.hourly/` 등.

cron이 실패하는 3대 원인입니다. 거의 모든 문제가 여기 해당합니다.

- ① **PATH가 다름** — cron의 PATH는 로그인 셸보다 훨씬 짧습니다. 명령을 절대 경로로 쓰거나 crontab 상단에 `PATH=` 를 선언하세요.
- ② **환경 변수 없음** — `.bashrc` 는 읽지 않습니다. 필요한 변수는 스크립트 안에서 직접 설정하거나 `source` 하세요.
- ③ **% 문자** — cron에서 % 는 개행으로 해석됩니다. `date +%Y%m%d` 는 `date +%Y\%m\%d` 로 이스케이프해야 합니다.

디버깅: `* * * * * /path/script.sh >> /tmp/cron.log 2>&1` 로 출력을 남겨 확인하세요.

5.3 접근 제어

```
/etc/cron.allow      # 존재하면 여기 적힌 사용자만 허용
/etc/cron.deny       # allow가 없을 때 여기 적힌 사용자만 차단
```

5.4 at — 일회성 예약

```
echo "/usr/local/bin/deploy.sh" | at 02:00
at now + 1 hour
at 14:30 tomorrow

atq                # 예약 목록
atrm 3             # 3번 작업 취소
sudo systemctl enable --now atd # atd가 떠 있어야 동작
```

5.5 systemd timer

cron의 현대적 대안입니다. 로그가 journal에 통합되고 의존성 관리가 가능합니다.

```
sudo vim /etc/systemd/system/backup.service
```

```
[Unit]
Description=Nightly backup

[Service]
Type=oneshot
ExecStart=/usr/local/bin/backup.sh
```

```
sudo vim /etc/systemd/system/backup.timer
```

```
[Unit]
Description=Run backup daily at 3am
```

```
[Timer]
OnCalendar=*-*-* 03:00:00
Persistent=true
RandomizedDelaySec=300

[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer
systemctl list-timers          # 다음 실행 시각 확인
journalctl -u backup.service  # 실행 로그
```

지시어	효과
OnCalendar=	절대 시각 지정
OnBootSec=	부팅 후 경과 시간
Persistent=true	꺼져 있어 놓친 실행을 부팅 후 보충
RandomizedDelaySec=	무작위 지연. 다수 서버의 동시 실행 분산

Persistent=true 가 cron 대비 큰 장점입니다. cron은 서버가 꺼져 있던 시간의 작업을 그냥 건너뛰지만, timer는 다음 부팅 때 실행합니다. 백업처럼 반드시 실행되어야 하는 작업에 적합합니다.

RandomizedDelaySec 도 유용합니다. 서버 100대가 정확히 03:00에 동시에 백업을 시작하면 스토리지와 네트워크가 마비됩니다.

6. 로그 분석 — journald와 rsyslog

6.1 journalctl

```
journalctl                # 전체 (오래된 것부터)
journalctl -e             # 끝으로 이동
journalctl -f             # 실시간 추적
journalctl -n 50          # 최근 50줄

journalctl -u nginx       # 특정 서비스
journalctl -u nginx -f    # 특정 서비스 실시간
journalctl -u nginx --since "1 hour ago"

journalctl -b             # 이번 부팅
journalctl -b -1          # 직전 부팅
journalctl --list-boots   # 부팅 이력

journalctl -p err         # 오류 이상만
journalctl -p warning..err # 범위 지정

journalctl --since "2026-09-01" --until "2026-09-02 12:00"
journalctl --since today

journalctl _PID=1234      # PID로 필터
journalctl _UID=1000      # 사용자로 필터
journalctl -k             # 커널 메시지만
journalctl -o json-pretty # JSON 출력 (파싱용)
```

우선순위(priority)

0 emerg	1 alert	2 crit	3 err
4 warning	5 notice	6 info	7 debug

장애 대응 3종 세트를 외워주세요.

- ① systemctl --failed — 실패한 서비스 목록
- ② journalctl -p err -b — 이번 부팅의 오류 전체
- ③ journalctl -u 서비스 -n 100 --no-pager — 해당 서비스 최근 로그

이 세 개로 대부분의 원인이 좁혀집니다.

6.2 저널 영속화

기본 설정에서 저널은 /run (메모리)에 저장되어 **재부팅하면 사라집니다**. 디스크에 남기려면:

```
sudo mkdir -p /var/log/journal
sudo systemd-tmpfiles --create --prefix /var/log/journal
sudo systemctl restart systemd-journald
```

```
sudo vim /etc/systemd/journald.conf
```

```
[Journal]
Storage=persistent
SystemMaxUse=1G
MaxRetentionSec=1month
```

```
journalctl --disk-usage      # 사용량
sudo journalctl --vacuum-size=500M # 크기로 정리
sudo journalctl --vacuum-time=7d  # 기간으로 정리
```

6.3 rsyslog와 전통 로그 파일

경로	내용
/var/log/syslog (Debian)	전반적인 시스템 로그
/var/log/messages (RHEL)	위와 동일
/var/log/auth.log / secure	인증·sudo 기록
/var/log/kern.log	커널
/var/log/nginx/	애플리케이션별 로그

원격 로그 전송

```
sudo vim /etc/rsyslog.d/50-remote.conf
```

```
*.* @192.168.10.50:514      # @@ = TCP, @ = UDP
```

로그를 중앙 서버로 모으는 이유는 두 가지입니다. ① 서버가 침해당하면 공격자가 로컬 로그를 지웁니다. 원격 사본이 있어야 추적이 가능합니다. ② 서버 수십 대의 로그를 시간순으로 대조하려면 한곳에 모여야 합니다. 이때 **NTP 동기화가 전제 조건**입니다.

6.4 logrotate

```
sudo vim /etc/logrotate.d/myapp
```

```
/var/log/myapp/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 appsvc appsvc
    sharedscripts
    postrotate
        systemctl reload myapp > /dev/null 2>&1 || true
    endscript
}
```

```
sudo logrotate -d /etc/logrotate.d/myapp # 시뮬레이션 (실행 안 함)
sudo logrotate -f /etc/logrotate.d/myapp # 강제 실행
```

postrotate 에서 서비스에 재개 신호를 주지 않으면, 프로세스가 이미 이름이 바뀐 옛 파일에 계속 쓰기 때문에 디스크 공간이 회수되지 않습니다. 9장의 "삭제됐지만 열려 있는 파일" 문제와 같은 원인입니다. copytruncate 를 쓰는 방법도 있지만 회전 순간의 로그가 유실될 수 있습니다.

7. 프로세스 관리

7.1 조회

```
ps aux                # 전체 프로세스 (BSD 스타일)
ps -ef                # 전체 (System V 스타일)
ps aux --sort=-%mem | head # 메모리 사용 상위
ps aux --sort=-%cpu | head # CPU 사용 상위
ps -eo pid,ppid,user,%cpu,%mem,etime,cmd --sort=-%cpu | head

pstree -p             # 트리 구조
pgrep -a nginx        # 이름으로 PID 찾기
pidof nginx
```

7.2 실시간 모니터링

```
top                # 기본
htop               # 개선판 (설치 필요, 권장)
```

top 안에서

```
P  CPU 순 정렬      M  메모리 순 정렬
l  CPU 코어별 표시  k  프로세스 종료
u  사용자 필터      q  종료
```

load average 읽기

```
load average: 2.15, 1.80, 1.45
              1분   5분   15분
```

이 값은 **실행 중이거나 대기 중인 프로세스** 수입니다. **CPU 코어 수와 비교**해야 의미가 있습니다. 4코어에서 load 4는 100% 활용이고, load 8은 과부하입니다.

리눅스의 load average는 CPU 대기뿐 아니라 **디스크 I/O 대기(D 상태)**도 포함합니다. CPU는 한가한데 load만 높다면 스토리지 병목을 의심하세요. 이때 `iostat -xz 2`로 넘어갑니다.

7.3 종료

```
kill 1234           # SIGTERM(15) — 정상 종료 요청
kill -9 1234         # SIGKILL — 강제 종료
kill -HUP 1234       # SIGHUP(1) — 설정 재읽기
killall nginx        # 이름으로 전부
pkill -u jihoon       # 사용자의 모든 프로세스
```

시그널	번호	특징
SIGTERM	15	정리 후 종료. 기본값, 우선 시도
SIGKILL	9	즉시 강제. 무시 불가. 데이터 손상 위험
SIGHUP	1	설정 재읽기
SIGSTOP / SIGCONT	19 / 18	일시정지 / 재개

kill -9를 습관적으로 쓰지 마세요. SIGKILL은 프로세스에 정리할 기회를 주지 않아 임시 파일이 남고, DB라면 데이터가 손상될 수 있습니다. 순서는 `kill` → 수 초 대기 → 안 되면 `kill -9`입니다.

D 상태(uninterruptible sleep) 프로세스는 `kill -9` 로도 죽지 않습니다. NFS 응답 대기나 디스크 오류가 원인이며, 근본 원인을 해결해야 합니다.

7.4 우선순위

```
nice -n 10 ./heavy_job.sh      # 낮은 우선순위로 실행
sudo renice -n -5 -p 1234      # 실행 중 변경
```

`nice` 값은 -20(최고)부터 19(최저)입니다. **음수로 낮추려면 root 권한이** 필요합니다. 백업이나 배치 작업을 `nice 10` 으로 돌리면 운영 서비스에 영향을 줄이면서 자원을 활용할 수 있습니다.

7.5 백그라운드 실행

```
./script.sh &                 # 백그라운드
jobs                           # 작업 목록
fg %1                           # 포그라운드로
bg %1                           # 일시정지
Ctrl+Z                          # 일시정지

nohup ./script.sh &           # 로그아웃해도 유지
```

장시간 작업은 `nohup` 보다 **tmux** 나 **screen** 을 쓰세요. SSH가 끊겨도 세션이 살아 있고, 다시 접속해 `tmux attach` 로 진행 상황을 그대로 볼 수 있습니다. 원격에서 큰 마이그레이션이나 컴파일을 돌릴 때 필수입니다.

8. 시스템 리소스 모니터링

8.1 CPU

```
lscpu                # CPU 정보 (코어 수 확인)
mpstat -P ALL 2      # 코어별 사용률
vmstat 2             # 종합 통계
uptime              # load average
```

vmstat 주요 열

열	의미	주의
r	실행 대기 프로세스	코어 수 초과 시 CPU 병목
b	I/O 대기 프로세스	지속되면 스토리지 병목
si / so	스왑 in/out	0이 아니면 메모리 부족
wa	I/O 대기 CPU 비율	높으면 디스크 확인
st	steal time	가상화 환경에서 호스트에 뺏긴 시간

클라우드/VM에서 st (steal time)가 높다면 **내 서버 문제가 아니라 호스트가 과밀한** 것입니다. 인스턴스 타입 변경이나 재배포를 검토해야 합니다. 애플리케이션을 아무리 튜닝해도 해결되지 않습니다.

8.2 메모리

```
free -h
cat /proc/meminfo
sudo smem -r          # 프로세스별 실제 사용량 (설치 필요)
```

```
Mem:              total    used    free   shared  buff/cache   available
               15Gi    4.2Gi    1.1Gi    120Mi     9.9Gi         10Gi
```

free 가 적다고 메모리 부족이 아닙니다. 리눅스는 남은 메모리를 파일 캐시(buff/cache)로 적극 활용하며, 애플리케이션이 요구하면 즉시 반환합니다.

실제로 봐야 할 값은 **available** 입니다. 이 값이 충분하면 문제없습니다. 캐시를 메모리 낭비로 오해해 drop_caches 를 실행하는 것은 오히려 성능을 떨어뜨립니다.

OOM Killer 확인

```
dmesg -T | grep -i "out of memory"
journalctl -k | grep -i oom
grep -i "killed process" /var/log/syslog
```

프로세스가 이유 없이 사라졌다면 OOM Killer를 의심하세요. 메모리가 고갈되면 커널이 점수(oom_score)가 높은 프로세스를 강제 종료합니다.

8.3 종합 도구

```
sudo apt install sysstat
sar -u 2 5          # CPU
sar -r 2 5          # 메모리
sar -n DEV 2 5      # 네트워크
sar -u -f /var/log/sysstat/sa02  # 과거 데이터 조회
```

sar 의 진짜 가치는 **과거 데이터 조회**입니다. "어젯밤 3시에 무슨 일이 있었나"는 top 으로는 알 수 없지만 sar 는 답합니다. sysstat 설치 후 수집을 할

성화해두면 사후 분석이 가능해집니다.

9. 패키지 관리

9.1 APT (Debian/Ubuntu)

```
sudo apt update          # 패키지 목록 갱신
sudo apt upgrade         # 업그레이드
sudo apt install nginx
sudo apt remove nginx    # 설정 파일은 남김
sudo apt purge nginx      # 설정까지 제거
sudo apt autoremove      # 불필요한 의존성 정리

apt search keyword
apt show nginx
apt list --installed
dpkg -l | grep nginx
dpkg -L nginx            # 설치된 파일 목록
dpkg -S /usr/sbin/nginx  # 이 파일은 어느 패키지 것인가
```

9.2 DNF/YUM (RHEL 계열)

```
sudo dnf check-update
sudo dnf update
sudo dnf install nginx
sudo dnf remove nginx

dnf search keyword
dnf info nginx
dnf list installed
rpm -qa | grep nginx
rpm -ql nginx           # 파일 목록
rpm -qf /usr/sbin/nginx # 소속 패키지
dnf history              # 작업 이력
sudo dnf history undo 15 # 되돌리기
```

dnf history undo 는 APT에 없는 강력한 기능입니다. 패키지 작업 후 문제가 생기면 트랜잭션 단위로 되돌릴 수 있습니다.

apt update 와 apt upgrade 는 다릅니다. update 는 **목록만** 갱신하고 아무것도 설치하지 않습니다. 실제 업그레이드는 upgrade 입니다. 항상 update 를 먼저 실행해야 최신 버전을 받습니다.

운영 서버에서 apt upgrade 를 무심코 실행하면 커널이나 서비스가 재시작될 수 있습니다. apt list --upgradable 로 목록을 먼저 확인하고, 보안 업데이트만 적용하려면 unattended-upgrade 설정을 검토하세요.

10. 커널 파라미터와 모듈

10.1 sysctl — 런타임 파라미터

```
sysctl -a                # 전체 조회
sysctl net.ipv4.ip_forward # 개별 조회
sudo sysctl -w net.ipv4.ip_forward=1 # 임시 변경 (재부팅 시 사라짐)

# 영구 설정
sudo vim /etc/sysctl.d/99-custom.conf
```

```
net.ipv4.ip_forward = 1
vm.swappiness = 10
fs.file-max = 2097152
net.core.somaxconn = 1024
net.ipv4.tcp_max_syn_backlog = 4096
```

```
sudo sysctl --system # 모든 설정 파일 재적용
sudo sysctl -p /etc/sysctl.d/99-custom.conf
```

/proc/sys/ 아래 파일과 1:1 대응합니다. net.ipv4.ip_forward 는 /proc/sys/net/ipv4/ip_forward 입니다.

10.2 커널 모듈

```
lsmod                # 적재된 모듈 목록
modinfo e1000        # 모듈 정보
sudo modprobe br_netfilter # 적재
sudo modprobe -r br_netfilter # 제거

# 부팅 시 자동 적재
echo "br_netfilter" | sudo tee /etc/modules-load.d/k8s.conf

# 모듈 차단
echo "blacklist nouveau" | sudo tee /etc/modprobe.d/blacklist-nouveau.conf
sudo update-initramfs -u
```

쿠버네티스 노드 구성에서 br_netfilter , overlay 모듈 적재와 net.bridge.bridge-nf-call-iptables=1 설정이 사전 요구사항입니다. 이 두 절의 내용이 실제로 쓰이는 대표 사례입니다.

11. SELinux

11.1 개념

기존 퍼미션(DAC)이 "누가 소유했는가"로 판단한다면, SELinux(MAC)는 "이 프로세스가 이 자원에 접근해도 되는 정책인가"를 추가로 검사합니다. root조차 정책을 벗어나면 차단됩니다.

11.2 모드

모드	동작
enforcing	정책 위반을 차단하고 기록
permissive	차단하지 않고 기록만. 디버깅용
disabled	완전 비활성

```

getenforce          # 현재 모드
sestatus            # 상세 상태
sudo setenforce 0   # permissive로 임시 전환
sudo setenforce 1   # enforcing으로 복귀

sudo vim /etc/selinux/config # 영구 설정

```

문제가 생겼다고 SELinux를 **disabled** 로 끄는 것은 최악의 선택입니다. 보안 계층 하나를 통째로 포기하는 것이고, 다시 켤 때 전체 재라벨링이 필요해 부팅이 매우 오래 걸립니다.

올바른 절차: **permissive** 로 전환 → 차단 로그 수집 → 필요한 정책이나 컨텍스트 수정 → **enforcing** 복귀.

11.3 컨텍스트

```

ls -Z /var/www/html/ # 파일 컨텍스트
ps -eZ | grep nginx  # 프로세스 컨텍스트

```

```

system_u:object_r:httpd_sys_content_t:s0
 ①      ②      ③      ④

```

① 사용자 ② 역할 ③ 타입 (가장 중요) ④ 레벨. 실무에서는 대부분 *타입*만 다룹니다.

```

# 임시 변경
sudo chcon -t httpd_sys_content_t /srv/web/index.html

# 영구 규칙 등록 후 적용 (권장)
sudo semanage fcontext -a -t httpd_sys_content_t "/srv/web(/.*)?"
sudo restorecon -Rv /srv/web

```

chcon 은 임시입니다. restorecon 이 실행되거나 재라벨링이 일어나면 원래대로 돌아갑니다. 영구적으로는 **semanage fcontext** 로 규칙을 등록한 뒤 **restorecon** 으로 적용해야 합니다. 이 차이를 모르면 "고쳤는데 다시 안 된다"를 반복하게 됩니다.

11.4 불린(boolean)

```

getsebool -a | grep httpd
sudo setsebool -P httpd_can_network_connect on

```

-P 는 영구 적용입니다. 웹 서버가 백엔드나 DB로 나가는 연결이 SELinux에 막히는 경우가 흔한데, 위 불린 하나로 해결되는 경우가 많습니다.

11.5 차단 로그 분석

```

sudo ausearch -m AVC -ts recent
sudo grep AVC /var/log/audit/audit.log | tail
sudo sealert -a /var/log/audit/audit.log # 해결책까지 제안

```

"설정에는 다 맞는데 서비스가 접근을 못 한다"면 RHEL 계열에서는 SELinux를 의심하세요. 특히 기본 경로가 아닌 곳(/srv , /data)에 웹 콘텐츠나 DB 파일을 두거나, 서비스 포트를 기본값에서 바꿨을 때 자주 발생합니다. 포트 변경은 **semanage port -a -t http_port_t -p tcp 8888** 로 등록합니다.

12. 셸 스크립트 기초

12.1 기본 구조

```
#!/bin/bash
set -euo pipefail

# 변수
BACKUP_DIR="/backup"
DATE=$(date +%Y%m%d)

# 사용
echo "백업 위치: ${BACKUP_DIR}/${DATE}"
```

set -euo pipefail 은 모든 운영 스크립트의 첫 줄에 넣으세요.

- -e — 명령이 실패하면 즉시 중단
- -u — 정의되지 않은 변수 사용 시 오류 (rm -rf \$DIR/ 사고 방지)
- -o pipefail — 파이프 중간이 실패해도 감지

이게 없으면 스크립트가 실패를 무시하고 계속 진행해 더 큰 피해를 만듭니다. 백업이 실패했는데 성공으로 보고하는 상황이 대표적입니다.

12.2 조건문과 test

```
if [ -f "$FILE" ]; then
    echo "파일 존재"
elif [ -d "$FILE" ]; then
    echo "디렉터리"
else
    echo "없음"
fi
```

파일 검사	의미	비교	의미
-f	일반 파일	-eq	숫자 같음
-d	디렉터리	-ne	숫자 다름
-e	존재	-lt / -gt	미만 / 초과
-r / -w / -x	읽기/쓰기/실행 가능	= / !=	문자열 비교
-s	크기가 0보다 큼	-z / -n	빈 문자열 / 비어있지 않음

변수는 항상 큰따옴표로 감싸세요. [-f \$FILE] 에서 \$FILE 이 비어 있거나 공백을 포함하면 문법 오류가 나거나 예상과 다르게 동작합니다. [-f "\$FILE"] 가 올바릅니다.

숫자 비교에 -eq 대신 == 를 쓰면 문자열 비교가 되어 "10" 과 "010" 이 다르다고 판정됩니다.

12.3 반복문

```
# 목록 순회
for f in /var/log/*.log; do
    echo "처리: $f"
done

# 횟수
for i in {1..5}; do
    echo "시도 $i"
```

```
done

# 파일을 줄 단위로 읽기 (안전한 방식)
while IFS= read -r line; do
    echo "$line"
done < input.txt
```

12.4 함수와 종료 코드

```
log() {
    echo "[$(date '+%F %T')] $*"
}

check_service() {
    local svc="$1"
    if systemctl is-active --quiet "$svc"; then
        log "$svc 정상"
        return 0
    else
        log "$svc 비정상"
        return 1
    fi
}

check_service nginx || exit 1
```

종료 코드는 `$?` 로 확인합니다. **0이 성공**, 그 외가 실패입니다. 일반 프로그래밍 언어의 참/거짓과 반대라 헷갈리기 쉽습니다.

12.5 실전 예 — 백업 스크립트

```
#!/bin/bash
set -euo pipefail

SRC="/srv/app"
DEST="/backup"
DATE=$(date +%V%m%d_%H%M%S)
KEEP_DAYS=14
LOG="/var/log/backup.log"

log() { echo "[$(date '+%F %T')] $*" | tee -a "$LOG"; }

# 사전 점검
[ -d "$SRC" ] || { log "오류: 원본 없음 $SRC"; exit 1; }
mkdir -p "$DEST"

# 백업 수행
log "백업 시작: $SRC"
tar -czf "${DEST}/app_${DATE}.tar.gz" -C "$(dirname "$SRC")" "$(basename "$SRC")"
log "백업 완료: app_${DATE}.tar.gz"

# 오래된 백업 정리
find "$DEST" -name "app_*.tar.gz" -mtime "+${KEEP_DAYS}" -delete
log "${KEEP_DAYS}일 이전 백업 정리 완료"
```

운영 스크립트에는 최소 세 가지가 있어야 합니다. ① **사전 점검** — 원본이 있는가, 디스크 여유가 있는가 ② **로그** — 언제 무엇을 했는지 ③ **실패 시 중단** — `set -e` 와 명시적 종료 코드. 세 가지가 없는 백업 스크립트는 조용히 실패하고 있을 가능성이 큼니다.

12.6 디버깅

```
bash -n script.sh    # 문법만 검사 (실행 안 함)
bash -x script.sh    # 각 명령을 출력하며 실행
set -x               # 스크립트 중간부터 추적
```

```
set +x          # 추적 종료  
  
shellcheck script.sh  # 정적 분석 도구 (강력 권장)
```

13. 실습 체크리스트 / 자주 하는 실수

13.1 실습 체크리스트

1. `systemd-analyze blame` 으로 부팅 시간을 잡아먹는 서비스 확인
2. GRUB에서 `e` 를 눌러 커널 파라미터를 임시 추가하고 부팅
3. 테스트 VM에서 `init=/bin/bash` 로 root 비밀번호 복구 실습
4. 커스텀 `.service` 유닛을 작성하고 `enable --now` 로 기동
5. `daemon-reload` 를 빠뜨렸을 때 변경이 반영되지 않는 것 확인
6. `Type=simple` 과 `forking` 을 바꿔 상태 표시가 달라지는 것 관찰
7. `systemctl edit` 으로 드롭인 재정의 후 `systemctl cat` 으로 병합 결과 확인
8. `disable` 과 `mask` 의 차이를 수동 `start` 로 검증
9. cron에 `date +%Y` 를 이스케이프 없이 넣어 실패를 재현한 뒤 수정
10. cron 작업의 PATH를 `env > /tmp/cronenv` 로 덤프해 로그인 셸과 비교
11. systemd timer를 만들고 `list-timers` 로 다음 실행 시각 확인
12. 저널 영속화를 설정하고 재부팅 후 `journalctl -b -1` 이 남는지 확인
13. logrotate를 `-d` 로 시뮬레이션한 뒤 `-f` 로 강제 실행
14. 메모리를 소진시켜 OOM Killer 동작을 `dmesg` 에서 확인 (테스트 VM에서만)
15. `nice` 로 우선순위를 바꾼 프로세스를 `top` 에서 비교
16. `sysctl -w` 로 임시 변경 후 재부팅으로 사라지는 것, `sysctl.d` 로 유지되는 것 비교
17. (RHEL) 웹 콘텐츠를 `/srv` 에 두고 SELinux 차단을 `ausearch` 로 확인 후 `semanage` 로 해결
18. `set -euo pipefail` 유무에 따라 스크립트가 실패 후 계속 진행하는지 비교
19. `shellcheck` 로 기존 스크립트를 검사해 지적 사항 확인

13.2 자주 하는 실수

실수	결과	올바른 방법
유닛 수정 후 <code>daemon-reload</code> 누락	변경 미반영	수정 후 항상 실행
유닛 파일을 <code>/lib</code> 에서 직접 수정	업데이트 시 소실	<code>systemctl edit</code> 드롭인
<code>ExecStart</code> 재정의 시 초기화 누락	중복 실행 오류	<code>ExecStart=</code> 먼저
Type 설정 오류	상태 오프시, 기동 실패	포그라운드=simple
<code>grub.cfg</code> 직접 편집	커널 업데이트 시 소실	<code>/etc/default/grub</code> + 재생성
cron에서 상대 경로·환경변수 의존	조용히 실패	절대 경로, PATH 명시
cron의 <code>%</code> 미이스케이프	명령 잘림	<code>\%</code>
cron 출력 미수집	실패 원인 불명	<code>>> log 2>&1</code>
저널 영속화 미설정	재부팅 시 로그 소실	<code>Storage=persistent</code>
logrotate에 reload 누락	디스크 공간 미반환	<code>postrotate</code> 추가
<code>kill -9</code> 남용	데이터 손상	SIGTERM 먼저
<code>free</code> 의 캐시를 부족으로 오해	불필요한 조치	<code>available</code> 확인
load average를 코어 수와 대조 안 함	오진	<code>lscpu</code> 로 코어 수 확인
<code>apt update</code> 만 하고 <code>upgrade</code> 로 착각	업데이트 미적용	둘은 다른 명령

SELinux를 <code>disabled</code> 로 회피	보안 계층 상실	permissive → 정책 수정
<code>chcon</code> 만으로 영구 기대	재라벨링 시 원복	<code>semanage fcontext</code>
스크립트에 <code>set -e</code> 없음	실패 후 계속 진행	<code>set -euo pipefail</code>
변수에 따옴표 누락	공백·빈값 오동작	<code>"\$VAR"</code>

13.3 장애 대응 기본 순서

1. `systemctl --failed` — 실패한 서비스가 있는가
2. `journalctl -p err -b` — 이번 부팅의 오류
3. `df -h` / `df -i` — 디스크와 inode 여유
4. `free -h` — available 확인
5. `uptime` + `lscpu` — load를 코어 수와 대조
6. `iostat -xz 2` — I/O 병목 여부
7. `ss -tulnp` — 서비스가 리스닝 중인가
8. `dmesg -T` | `tail -50` — 하드웨어·커널 이상

학습용으로 새로 정리한 자료입니다. 명령과 설정은 배포판·버전에 따라 다를 수 있으니 실제 적용 전 `man` 페이지와 공식 문서로 확인하시기 바랍니다.